Programmatoid and neuronoid computations

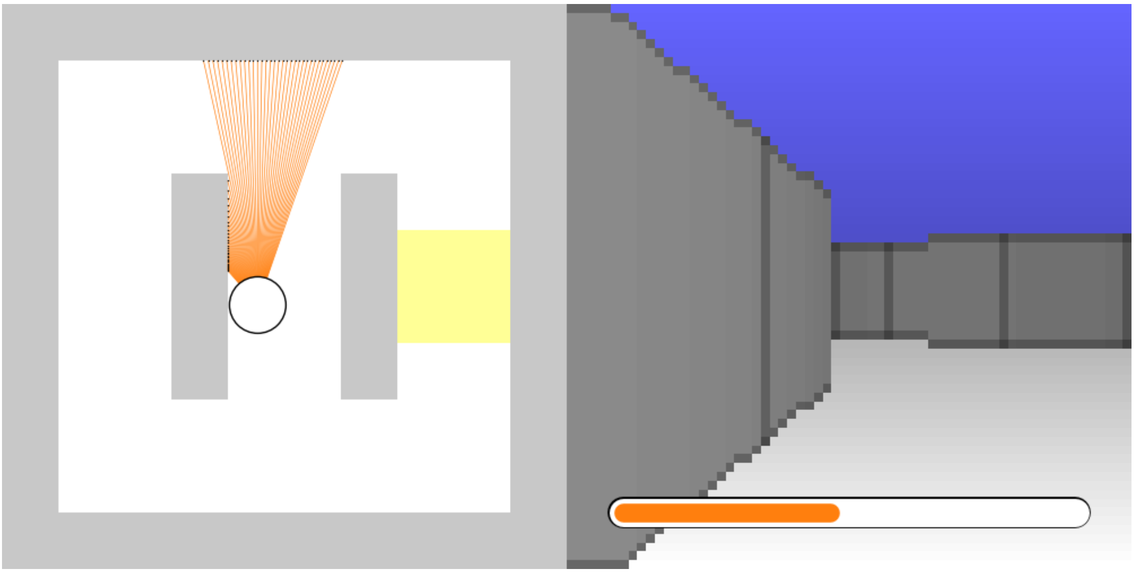
Reference document, v1

Contents

1 The braincraft challenge

Let us consider the following digital experimental setup, as described in the braincraft challenge presentation[1].

1.1 Simplified problem statement



As shown in this figure[2], the braincraft challenge[3] bot moves at a constant with sensor inputs and one orientation output, it uses some energy and refill this energy on a given yellow location. The 2D space size is $[0,1] \times [0,1]$. The bot starts in the middle and is oriented upward.

Input variables	
$p_l \in]0_{\text{no-wall}}, 1_{\text{wall-hit}}]$	Leftward wall proximity, at +30° on the left.
$p_r \in]0_{\text{no-wall}}, 1_{\text{wall-hit}}]$	Rightward wall proximity, at -30° on the right.
$p_a \in]0_{\text{no-wall}}, 1_{\text{wall-hit}}]$	Ahead wall proximity, at 0°.
$c_{l\bullet} \in \{0,1\}, \bullet \in \{b_{\text{blue}}, r_{\text{red}}\}$	Leftward binary red and blue color detectors.
$c_{r\bullet} \in \{0,1\}, \bullet \in \{b_{\text{blue}}, r_{\text{red}}\}$	Rightward binary red and blue color detectors.
$g_e \in [0_{\text{death}}, 1_{\text{full}}]$	Energy gauge value.

Output variables	
$d_l \in [0, 1_{+5^o}]$	Left orientation difference.
$d_r \in [0, 1_{-5^o}]$	Right orientation difference.

With respect to the original braincraft challenge:
- three leftward, ahead, and rightward external sensors only are considered,
- color is input as binary variables, and always available,
- the output stands on positive normalized signal,
- the wall hit indicator is not used, and wall hits are ignored,
- the heuristics consider the energy source as constant and unbounded.

[1] `https://github.com/rougier/braincraft/blob/master/README.md#introduction`
[2] Shared here by courtesy of Nicolas P. Rougier.
[3] `https://github.com/rougier/braincraft`

Neglecting the energy source bound, leak and refill mechanism, and ignoring wall hits yields sub-optimal heuristics, but the goal is not to "win" but the challeng, but to implement programmatoid and neuronoid mechanisms.

1.2 Preprocessing

1.2.1 Proximity sensors

Motivation Simplifies the navigation by using a simple triplet of input.

Implementation
- We now simply considered the frontal and the two lateral sensors of maximal angle.
- A simple sum or average could also be used, or any linear combination of the input in afferent units.
- A softmax mechanism could also be used, as derived in the sequel, enjoying a neuronoid approximation, and allowing to balance between averaging and computing the maximum.

1.2.2 Color sensors

Motivation The color blob detection is to perform either on the left or on the right, allowing the color input to be compacted. For each color, a channel is specified, simplifying the programmatoid implementation and providing an input closer to biological colored vision. The setup original color input is a discrete color index. The preprocessed color output is binary, accounting for the presence, or not, of a least one related color index.

Implementation For the distributed implementation, each camera color index value is mapped on each color channel input with the 0 value if the color is different and the 1 value if equal, e.g., using step unit:

$$c_{\dagger\bullet} \leftarrow \text{ if } \mathrm{Or}_{k \in K_\dagger}(i_\bullet - c[k]) = 0 \text{ then } 1 \text{ else } 0, \dagger \in \{l_{\text{left}}, r_{\text{right}}\}, \bullet \in \{b_{\text{blue}}, r_{\text{red}}\}$$

where $K_\dagger$ stands for the left or right sensor related indexes, $i_\bullet$ stands for the color index, and $c[k]$ stands for the sensor color index value.

1.2.3 Energy increase or decrease

Motivation In order to detect if the bot has been reefed, we must save the energy value at different moment, and compare the values, because among the trajectory the absolute value varies.

Implementation It uses two variables

$$\begin{array}{lll} g_{e1} \in [0,1], & g_{e1}|_{t=0} = 0 & \text{Energy after the last turn.} \\ g_{d1} \in [0,1], & g_{ed}|_{t=0} = 0 & \text{Energy difference, before the next turn, and after the last turn.} \end{array}$$

which are updated during the navigation as detailled in the next section.

1.2.4 Output unsigned normalization

Motivation As far as biologically plausible signals related to neuronal firing rate is concerned, we better consider only positive value, normalized by the maximal firing rate.

Implementation This can be considered as a minimal output linear unit:

$$O \leftarrow 5\,(d_l - d_r)$$

1.3 A generic controller

1.3.1 Navigation

Heuristic : The bot runs at constant velocity,

(i) ahead by default, thanks to a linear correction, high enough to correct the direction, small enough to avoid oscillations, and either

(ii) performs a quarter-turn in the preferred orientation as soon as a wall is closed ahead, or

(ii) performs a about-turn if the energy has been increased by an energy source.

With this about-turn heuristic, the bot remains in the corridor with feeding source. However, to perform an about-turn the bot must hit the wall, with generates an energy loss, immedtaly compensated by refilling energy on the source. This however does not take into account the source energy is bounded and requires time to refill.

Note: The bot is blind with respect to what is on its sides because the field of view is too narrow. Therefore, forward proximity to the wall, and predefined delays are to be used.

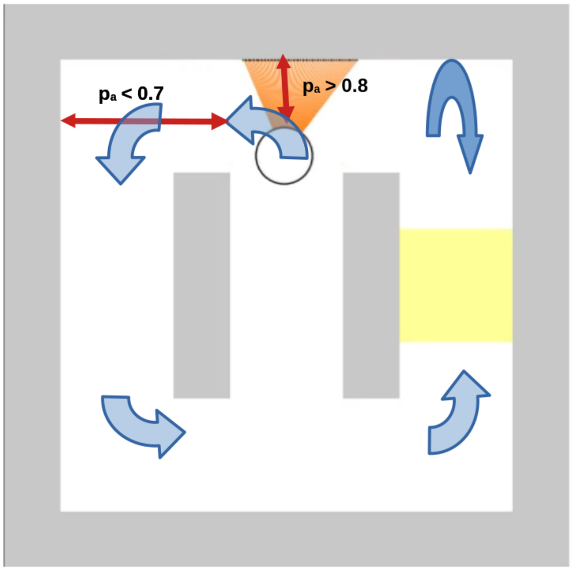


Figure 1: Illustrates the navigation implementation. The navigation is made of quarter-turns to find the source and then about-turns to remain around the source:

Quarter-turn is triggered by forward wall proximity and stops when this proximity is low enough. It is used to find the feeding source.

About-turn is triggered by forward wall proximity and stops after a fixed delay. It is used to remain around the feeding source.

Internal variables

$$\begin{array}{lll}
b_d \in \{0_{\text{rightward}}, 1_{\text{leftward}}\}, & b_d|_{t=0} = 0 & \text{Initial quarter-turn direction.} \\
b_t \in \{0_{\text{forward}}, 1_{\text{turning}}\}, & b_t|_{t=0} = 0 & \text{Forward versus turning mode.} \\
b_a \in \{0_{\text{quarter-turn}}, 1_{\text{about-turn}}\}, & b_t|_{t=0} = 0 & \text{Quarter-turn versus about-turn mode.}
\end{array}$$

Internal constants

$$\begin{array}{rcll}
\gamma & = & 0.035 & \text{Linear feedback gain in forward mode, maximize stability without oscillation.} \\
\alpha & = & 5.0 & \text{Maximal turn angle, allowing to perform a quarter turn in 18 time steps.} \\
\beta_1 & = & 0.8 & \text{Proximity threshold triggering a quater-turn.} \\
\beta_0 & = & 0.7 & \text{Proximity threshold ending a quater-turn.} \\
\delta & = & 36 & \text{About-turn duration in number of time steps.}
\end{array}$$

Programmatoid solution :

$$\begin{array}{rccc}
d_l & \leftarrow & \text{if } b_t = 0 \text{ then } \gamma\, p_r & \text{elif } b_d = 1 \text{ then } \alpha \text{ else } 0 \\
d_r & \leftarrow & \text{if } b_t = 0 \text{ then } \gamma\, p_l & \text{elif } b_d = 0 \text{ then } \alpha \text{ else } 0 \\
 & & \text{linear correction to} & \text{turning} \\
 & & \text{maintain direction ahead} & \text{left or right}
\end{array}$$

while the bot turning state implementation writes:

Wall proximity turn trigerring, if not yet done:
$b_1 \quad = \quad p_a > 0.8$ and not b_t
About-turn choice iff the energy increased:
$b_a \quad \leftarrow \quad$ if b_1 and $g_{e1} > 0$ then $g_e > g_{e1}$ else b_a
About-turn direction change:
$b_d \quad \leftarrow \quad b_1$ and b_a then $1 - b_d$ else b_d
About-turn delay, triggered at about-turn start:
Delay_0(b_w, b_1 and b_a, δ)
Turn end triggering:
$b_0 \quad = \quad b_t$ and if b_a then b_w else $p_a < \beta_0$
Turn mode detection:
$b_t \quad \leftarrow \quad$ if b_1 then 1 elif b_0 then 0 else b_t
Turn end detection:
$b_s \quad = \quad b_0$ and not b_1
Gauge energy update:
$g_{d1} \quad \leftarrow \quad$ if b_s then $g_e - g_{e1}$ else g_{d1}
$g_{e1} \quad \leftarrow \quad$ if b_s then g_e else g_{e1}

- The intermediate variables b_1, b_0 and b_s are not state variable but simply allows to write a more factorize and readable solution.
- It is important to note that the evaluations are to be performed in sequence, for the mechanism to be valid.
- As detailed in the sequel, the Delay_0($b_w, b_{\text{trigger}}, \delta$) construct defines a delay of δ time steps, triggered by b_{trigger} and which output b_w equals 0 during this delay.

1.3.2 Task 1: Simple decision

Heuristic Finds on which side is the energy loop and stay around the energy source until, when found.

This is given as a side effect of the navigation implementation, and it also works if the energy source location changes with time.

1.3.3 Task 2: Cued environment decision

Heuristic Restricts navigation to the open side with an energy source, as indicated by a blue color at the beggining.

Internal variable

$b_l \in \{0,1\} \quad b_l|_{t=0} = 0$ Blue detected on the left.
$b_r \in \{0,1\} \quad b_r|_{t=0} = 0$ Blue detected on the right.

Programmatoid solution

Detects, leftward or righward, the 1st blue cue.
$b_l \quad \leftarrow \quad$ if $b_l = 0$ and $b_r = 0$ and $c_{lb} = 1$ then1 else b_l
$b_r \quad \leftarrow \quad$ if $b_l = 0$ and $b_r = 0$ and $c_{rb} = 1$ then1 else b_r
Sets the left (1) or right (0) direction according to the cue.
$b_d \quad \leftarrow \quad$ if $c_{lb} = b_l$ then 1 elif $c_{rb} = b_r$ then 0 else b_d

1.3.4 Task 3: Valued environment decision

Heuristic In this case, the previous energy increase has to be compared with the present energy increase, in order to choose the largest. This simply modifies the evaluation of the about-turn trigger, as follows

Programmatoid solution

$$b_a \quad \leftarrow \quad \text{if } b_1 \text{ and } g_{d1} > 0 \text{ and } g_e > g_{e1} \text{ and } g_e - g_{e1} > g_{d1} \text{ else } b_a$$

2 Experimental validation

	task 1	task 2	task 3
programmatic			
programmaticoid			
programmatoid			
neuronoid			

3 Programmatoid computation

3.1 Definition of a programmatoid computation

We name "programmatoid computation" the conception of an input-output straight-line program[4] implementing an operator on either binary values $\in \{0,1\}$ or continuous normalized[5] values $\in [0,1]$, which evolution is defined using an LN-system with linear units or the step-function $\mathrm{H}(\cdot)$ as non-linearity, this writing:

$$o_n[t] = \left\{ \begin{array}{c} \mathrm{H}(z_n[t]) \\ \mathrm{Id}(z_n[t]) \end{array} \right. , z_n[t] \stackrel{\rm def}{=} \sum_{m \in \{1,M\}} w_{nm}\, i_m[t-1] + w_{n0}, n \in \{1,N\}$$

where $\mathrm{Id}(\cdot)$ stands for the identity function. Here

- $i_m[t]$ is the m-th input at a discrete t, $i_m[t]|_{t=0} = 0$ and
- $o_n[t]$ is the n-th output a time t

including recurrent system with some output corresponding to inputs, while

- $w_{nm}, n \in \{1,N\}, m \in \{0,M\}$ are the systems parameters, or, weights for $m > 0$ and bias for $m = 0$.

The step-function, also called Heaviside function, implements the computation of the sign of a value:

$$\begin{array}{rcl} \mathrm{H}(x) & \stackrel{\rm def}{=} & \left\{ \begin{array}{rcl} 1 & \mbox{if} & x > 0 \\ 1/2 & \mbox{if} & x = 0 \\ 0 & \mbox{if} & x < 0 \end{array} \right. \\ & = & \mbox{if } x > 0 \mbox{ then } 1 \mbox{ elif } x = 0 \mbox{ then } 1/2 \mbox{ else } 0. \end{array}$$

The design choice of $\mathrm{H}(0) = 1/2$, instead for instance $\mathrm{H}(0) = 0$, this latter simplifying some formula, is due the neuronoid approximation developed in the sequel.

By extension, we use, for any property $\mathcal{P}$, the notation:

$$\mathrm{H}(\mathcal{P}) \stackrel{\rm def}{=} \mbox{if } \mathcal{P} \mbox{ then } 1 \mbox{ else } 0$$

A "generalized programmatoid computation" may include other dedicated non-linearities, providing these are pure fonction[6], i.e., no side effect, same inpouts yield same output, and providing execution time does not depends on the input, while reasonably small; this will not used here.

For the sake of simplicity if the left hand size correspond to a value at time t and right-hand size to values at $t-1$, which will be always here, and if not informative, temporal indexes are omitted, i.e. the previous equation will be written:

$$o_n \leftarrow \mathrm{H}(\textstyle\sum_{m \in \{1,M\}} w_{nm}\, i_m + w_{n0}), n \in \{1,N\}$$

3.2 On sequential evaluation

By design, evaluation is done in sequence at a given t, i.e., from $o_1[t]$ to $o_N[t]$. Each time the evaluation order counts, this is made explicit in sequel,

However, it must be noted that it is not a limitation because in any programmatoid instruction sequence of the form:

[4] `https://en.wikipedia.org/wiki/Straight-line_program`

[5] An alternative would have be to consider continuous normalized values $\in [-1,1]$, with the sign function, and its mollification using $\tanh(\cdot)$:

$$\mathrm{Sg}(x) \stackrel{\rm def}{=} \mbox{if } x > 0 \mbox{ then } 1 \mbox{ elif } x < 0 \mbox{ then } -1 \mbox{ else } 0 = -1 + 2\,\mathrm{H}(x) = \lim_{\omega \rightarrow +\infty} \tanh(\omega\, x),$$

which would have yields to same general results, with somehow heavier derivations, as the reader can verify.

Implementing signed variable is obvious in a programmatoid environment:

$$s = s_+ - s_-, \left\{ \begin{array}{rcl} s_+ & \stackrel{\rm def}{=} & \mbox{if } s > 0 \mbox{ then } s \mbox{ else } 0 \\ s_- & \stackrel{\rm def}{=} & \mbox{if } s < 0 \mbox{ then } -s \mbox{ else } 0, \end{array} \right.$$

thus manipulating signed variables in a environment with only positive variables.

[6] `https://en.wikipedia.org/wiki/Pure_function`

$$\left\{ \begin{array}{lcl} & \cdots & \\ o_n[t] & = & f_n\left(\cdots i_m[t-1], \cdots, \cdots o_l[t-1] \cdots, \cdots o_k[t] \cdots\right), k < n \\ & \cdots & \end{array} \right.$$

recursively replacing all right-end side values of $o_k[t]$ by the $f_n(\cdot)$ correspondent value, will at the fixed point, yields a instruction set for which the evaluation order is meaningless, since interdependences between output variables has been canceled by substitution.

3.3 Comparison implementation

Given two numerical variables v_1 and v_2, using the notation , we have the equivalence[7], considering $true$ as 1 and $false$ as 0:

$\mathrm{H}(v_1 > v_2)$	$\mathrm{H}(v_1 \leq v_2)$	$\mathrm{H}(v_1 = v_2)$	$\mathrm{H}(v_1 \neq v_2)$	
not $\mathrm{H}(v_1 \leq v_2)$	$\mathrm{H}(\mathrm{H}(v_1 - v_2))$	$\mathrm{H}(v_1 \leq v_2)$ and $\mathrm{H}(v_2 \leq v_1)$	not $\mathrm{H}(v_1 = v_2)$	with $\mathrm{H}(0) = 1/2$
$\mathrm{H}(v_1 - v_2)$	not $\mathrm{H}(v_1 - v_2)$	not $\mathrm{H}(v_1 \neq v_2)$	$\mathrm{H}(v_1 > v_2)$ or $\mathrm{H}(v_2 > v_1)$	with $\mathrm{H}(0) = 0$

while $\mathrm{H}(v_1 < v_2) = \mathrm{H}(v_2 > v_1)$ and $\mathrm{H}(v_1 \geq v_2) = \mathrm{H}(v_2 \leq v_1)$. We thus can implement any numerical comparison as a programmatoid, providing that we can implement boolean expressions, as given now.

3.4 Boolean expression implementation

Given binary variables $b_n \in \{0_{false}, 1_{true}\}, n \in \{1, N\}$ we have the obvious[8] correspondence:

b_1	b_1 and b_2	b_1 or b_2	not b_1
$\mathrm{H}(b_1 - 1/2)$	$b_1\, b_2 = \mathrm{H}(b_1 + b_2 - 3/2)$	$\mathrm{H}(b_1 + b_2 - 1/2)$	$1 - b_1 = \mathrm{H}(1/2 - b_1)$

and more generally:

$$\begin{array}{rcl} \mathrm{and}_{n \in \{1,N\}} & = & \mathrm{H}\left(\sum_{n \in \{1,N\}} b_n - N + 1/2\right) = \prod_{n \in \{1,N\}} \mathrm{H}(b_n - 1/2) = \prod_{n \in \{1,N\}} b_n \\ \mathrm{or}_{n \in \{1,N\}} & = & \mathrm{H}\left(\sum_{n \in \{1,N\}} b_n - 1/2\right) \end{array}$$

An interesting consequence is that binary variable products can be translated to a programmatoid.

Let us also notice that all arguments x of the step function $\mathrm{H}(\cdot)$ still verify $|x| \geq 1/2$.

3.5 Conditional expression implementation

A conditional expression on variables on any numerical type $v_n, n \in \{0, 1\}$ with a binary variable $b_1 \in \{0, 1\}$ writes[9] :

$$\begin{array}{rcl} v & \leftarrow & \text{if } b_1 \text{ then } v_1 \text{ else } v_0 \\ & = & b_1\, v_1 + (1 - b_1)\, v_0 \end{array}$$

while, for binary variable, i.e., if and only if $v_n \in \{0, 1\}, n \in \{0, 1\}$:

$$\begin{array}{rcl} & = & \mathrm{H}(v_1 + b_1 - 3/2) + \mathrm{H}(v_0 + (1 - b_1) - 3/2) \\ & = & \mathrm{H}(v_1 + b_1 - 3/2) + \mathrm{H}(v_0 - b_1 - 1/2) \\ & = & \mathrm{H}(\mathrm{H}(v_1 + b_1 - 3/2) + \mathrm{H}(v_0 - b_1 - 1/2)) \end{array}$$

as easy to verify, using for instance a truth table. Here:
- products with $(1 - b_1)$ and b_1 behaves as switches between v_o and v_1
- when v_i are binary, we are left with a two layer computation, the first layer being built from two programmatoid, and the second layer from either another programmatoid or a simple linear unit, i.e., a linear combination of values.

This generalizes to conditional expressions on variables on any numerical type $v_n, n \in \{0, N\}$: [10]:

---

[7] Considering the pseudo truth table, with $\mathrm{H}(0) = 1/2$:

	$\mathrm{H}(v_1 > v_2)$	$\mathrm{H}(v_1 = v_2)$	$\mathrm{H}(v_1 < v_2)$
$\mathrm{H}(v_1 - v_2)$	1	1/2	0
$\mathrm{H}(\mathrm{H}(v_1 - v_2))$	1	1	0
$\mathrm{H}(\mathrm{H}(v_2 - v_1))$	0	1	1
$\mathrm{H}(\mathrm{H}(v_1 - v_2)) + \mathrm{H}(\mathrm{H}(v_2 - v_1)) - 1$	0	1	0

we obtain the expected results.

[8] Easy to verify with, e..g., a truth table, and by induction for the generalized formula.

[9] It corresponds to the standard languages conditional expression:

```
v = b_1 ?  v_1 :  v_0,
```

or the Python construct:

```
v = v_1 if v_1 else v_0.
```

[10] By induction, considering the second line, one one hand, due to the products, if $b_n = 0, n \in \{1, N\}$ we obtain v_0. On the other hand, if $b_k = 0, k < K$ and $b_K = 1$, due the products, we obtain v_K, which is precisely the semantic of the conditional

$$
\begin{array}{rcl}
v & \leftarrow & \text{if } b_1 \text{ then } v_1 \,[\text{ elif } b_n \text{ then } v_n]_{n\in\{2,N\}} \text{ else } v_0 \\
 & = & \prod_{n'\in\{1,N\}} (1-b_{n'})\, v_0 \\
 & + & \sum_{n\in\{1,N\}} b_n \prod_{\substack{n'\in\{1,N\},\\ n'\neq n}} (1-b_{n'})\, v_n \\
 & = & \sum_{n\in\{0,N\}} h_n\, v_n \\
\text{with } h_0 & \stackrel{\text{def}}{=} & \mathrm{H}(1/2 - \sum_{n'\in\{1,N\}} b_{n'}) \in \{0,1\} \\
\text{and } h_n & \stackrel{\text{def}}{=} & \mathrm{H}(b_n - \sum_{n'\in\{1,N\}, n'\neq n} b_{n'} - 1/2) \in \{0,1\}, n\in\{1,N\}
\end{array}
$$

while, for binary variable, i.e., if and only if $v_n \in \{0,1\}, n\in\{0,N\}$:

$$
\begin{array}{rl}
= & \mathrm{H}(v_0 - \sum_{n'\in\{1,N\}} b_{n'} - 1/2) \\
+ & \sum_{n\in\{1,N\}} \mathrm{H}(v_n + b_n - \sum_{n'\in\{1,N\}, n'\neq n} b_{n'} - 3/2).
\end{array}
$$

Let us also notice that all arguments x of the step function $\mathrm{H}(\cdot)$ again verify $|x| \geq 1/2$.

As a consequence,

- *A N-term conditional expression on binary variables reduces to a two layers programmatoid of N and 1 unit, the former unit being either linear of with a step-wise function.*
- *A N-term conditional expression on continuous variables reduces to a two layers system of N programmatoid and an output unit with h_n exclusive switches, as defined in the sequel.*

3.5.1 On conditional instruction implementation

Let us consider a construct of the form:

```
if condition then name = value endif
```

It is obviously equivalent to the conditional expression:

```
name = if condition then value else name
```

in words: the value is change if and only if the condition is true.

Any complex conditional instruction of the form:

```
if condition_1 then instruction_1 instructions_2
elif condition_2 then instructions_3
...
else instructions_4 endif
```

rewrites:

```
if condition_1 then instruction_1 endif
if condition_1 then instructions_2 endif
if not condition_1 and condition_2 then instructions_3 endif
...
if not condition_1 and not condition_2 then instructions_4 endif
```

in words: any conditional instruction decomposes into elementary conditional expressions.

Similarly fixed length loops can be easily expanded, thus can also be introduced as a syntactic sugar in straight-line pieces of codes, e.g. a construct of the form, for a bounded variable value `n_1`, $0 \leq$ `n_1` $\leq$ `n_max`

```
for n from 0 to n_1 ≤ n_max do instructions_1 endfor
```

rewrites:

```
if n ≥ 0 then substitute(n = 0, instructions_1)
...
if n = n_1 then substitute(n = n_1, instructions_1)
```

in words: expands the loop in sequence for any possible value, while if `n_1 > n_max` a compilation error is to be thrown.

All together, a combination of straight-line pieces of codes with conditional expression or fixed length loops, decomposes into a pure straight-line piece of code.

---

expression of the first line.
The 3rd line is deduced from the second line, transforming the binary products on the corresponding programmatoid while:

$$
\begin{array}{rcl}
h_0 & \stackrel{\text{def}}{=} & \mathrm{H}(\sum_{n'\in\{1,N\}}(1-b_{n'}) - N + 1/2) \\
 & = & \mathrm{H}(1/2 - \sum_{n'\in\{1,N\}} b_{n'}) \\
h_n & \stackrel{\text{def}}{=} & \mathrm{H}(b_n + \sum_{n'\in\{1,N\}, n'\neq n}(1-b_{n'}) - N + 1/2) \\
 & = & \mathrm{H}(b_n - \sum_{n'\in\{1,N\}, n'\neq n} b_{n'} - 1/2)
\end{array}
$$

The 4rd line integrates the v_n variables in the programmatoid sum, since they are binary, and provides the same obvious algebraic reduction as for the 3rd line.

4 Neuronoid computation

4.1 Neuronoid unit

By "neuronoid" we name the not very biologically plausible[11] simplest biological neuron or neuron small ensemble inspired by mean-field modeling[12] of the Hodgkin–Huxley neuronal axon model[13].

It is defined the equation:

$$\tau \frac{\partial v_i}{\partial t}(t) + v_i(t) = z_i(t), \quad z_i(t) \stackrel{\text{def}}{=} \mathrm{h}\left(\sum_j w_{ij}\, v_j(t) + w_{i0}\right),$$

$$\begin{array}{rcll} \mathrm{h}(x) & \stackrel{\text{def}}{=} & 1/\left(1+e^{-4\,x}\right) & \\ & = & (1+\tanh(2\,x))/2 = \mathrm{sech}(2\,x)\, e^{2\,x}/2 & \text{(tanh linear relation)} \\ & = & \mathrm{h}(x_0) + \mathrm{sech}(2\,x_0)^2\,(x-x_0) + O((x-x_0)^2), & \text{(local 1st order behavior)} \\ & = & 1 - e^{-4\,x} + O(e^{-8\,x}) & \text{(behavior toward infinity).} \end{array}$$

Here, $\mathrm{h}(\cdot)$ is the normalized sigmoid with:

$$\mathrm{h}(-\infty) = 0, \mathrm{h}(0) = 1/2, h'(0) = 1, \mathrm{h}(+\infty) = 1, \mathrm{h}(x) = 1 - \mathrm{h}(-x),$$

and is derived from from considerations on mean-field modeling [Cessac and Samuelides, 2007].

Following this weak analogy with biological neurons, v_i stands for the membrane potential, so that:

$$\begin{array}{rcll} v(t) & = & 1/\tau \int_0^t e^{-(t-s)/\tau}\, z(s)\, ds + v(0)\, e^{-t/\tau} & \\ & = & z(0) + e^{-t/\tau}\,(v(0) - z(0))\big|_{z(t)=z(0)} & \text{(constant input)} \\ & = & z(t)|_{\tau=0} & \text{(no leak),} \end{array}$$

and the corresponding discrete approximation using an Euler schema writes:

$$\begin{array}{rcll} & \simeq & (1-\gamma)\, v(t-\Delta t) + \gamma\, z(t-\Delta t)) & \\ & = & \sum_{s=0}^{t-1} \gamma\,(1-\gamma)^{t-s+1}\, z(s) + v(0)\,(1-\gamma)^t & \\ & = & z(0) + (1-\gamma)^t\,(v(0) - z(0))\big|_{z(t)=z(0)} & \text{(constant input)} \\ & = & z(t)|_{\gamma=1} & \text{(no leak).} \end{array}$$

with $0 < \gamma < 1$ and $0 < \tau$ in one-to-one correspondence:

$$\gamma \stackrel{\text{def}}{=} 1 - e^{\frac{1}{\tau}} \Leftrightarrow \tau = -\frac{1}{\log(1-\gamma)}, \text{while} \lim_{\gamma\rightarrow 0} \tau = +\infty, \lim_{\gamma \rightarrow 1} \tau = 0.$$

All this is just very standard derivations, available as `Maple` code[14], of the vanilla neuron model, as already proposed by [Lapicque, 1907].

The leak term, in the discrete case, is computationally equivalent to a recurrent linear connection: It will thus be implemented as such in the sequel.

4.2 Definition of a neuronoid computation

We call "neuronoid computation" the conception of an input-output transform based on feed-forward and recurrent combination with only neuronoids as non-linearities:

$$o_n[t] = \mathrm{h}(z_n[t]), z_n[t] \stackrel{\text{def}}{=} \sum_{m \in \{1,M\}} w_{nm}\, i_m[t-1] + w_{n0}, n \in \{1, N\}$$

with the same notations as for programmatoid computations, and with determistic evaluation sequence.

4.3 Step-function mollification

The step-function approximates sigmoid with a huge slope at zero, i.e.:

$$\forall x \neq 0, \mathrm{H}(x) = \lim_{\omega \rightarrow +\infty} \mathrm{h}(\omega\, x),\, h'(\omega\, x)|_{x=0} = \omega$$

while the convergence is also obtained for $v = 0$ in the distribution sense with $\mathrm{H}(0) = \mathrm{h}(0) = 1/2$. More precisely, the $\mathcal{L}_1$ error magnitude, on $]-\infty, +\infty[$, writes:

$$|\epsilon_{H,\omega}(x)|_{\mathcal{L}_1} = \frac{\log(2)}{2}\,\frac{1}{\omega}, \epsilon_{H,\omega}(x) \stackrel{\text{def}}{=} \mathrm{H}(x) - \mathrm{h}(\omega\, x)$$

while:

$$|\epsilon_{H,\omega}(x)| = e^{-4\,\omega} + O\left(e^{-8\,\omega}\right)$$

[11] https://en.wikipedia.org/wiki/Biological_neuron_model#Relation_between_artificial_and_biological_neuron_models

[12] https://inria.hal.science/cel-01095603v1

[13] https://en.wikipedia.org/wiki/Hodgkin-Huxley_model

[14] https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/neuronoid.mpl.out.txt

It has been noticed that, except for numerical comparisons, all arguments x of the step function $\mathrm{H}(\cdot)$ verify $|x| \geq 1/2$, and the related error is negligible as soon as, say, $\omega \geq 10$:

ω	1	2	5	10	20	50
$\epsilon_\omega(\pm 1/2)$	0.119	0.0180	$0.455\,10^{-4}$	$0.206\,10^{-10}$	$0.426\,10^{-19}$	$0.372\,10^{-45}$

- *Any programmatoid computation involving the $H(\cdot)$ function can thus be approximated by a neuronoid, with $\tau = 0$, and sufficiently large ω, with an exponentially decreasing residual.*

Derivations are available as `Maple` code[15].

4.4 Approximation of the identify function

We also have to use a the linear part sigmoid to approximate the identity function $\mathrm{Id}(x) = x$ defining:

$$\begin{array}{rcl} \mathrm{Id}_{\omega'}(x) & \stackrel{\mathrm{def}}{=} & \omega'\,(\mathrm{h}(x/\omega') - 1/2) \\ & = & x - \frac{4}{3}\frac{1}{\omega'^2}\,x^3 + O\left(x^5\right) \end{array}$$

and writing $\epsilon_{l,\omega'}(x) \stackrel{\mathrm{def}}{=} x - l_{\omega'}(x)$, the $\mathcal{L}_1$ error magnitude on $[-M, +M]$ writes:

$$\int_{x=-M}^{x=+M} |\epsilon_{l,\omega'}(x)|\,dx = \frac{2}{3}\,\frac{M^4}{\omega'^2} + O\left(\frac{1}{\omega'^4}\right)$$

and the related $\mathcal{L}_0$ error magnitude:

$$\max(|\epsilon_{l,\omega'}(x)|), x \in [-M, M] = \epsilon_{l,\omega'}(M) = \frac{4}{3}\,\frac{M^3}{\omega'^2} + O\left(\frac{1}{\omega'^4}\right)$$

In a positive interval, by symmetry, the $\mathcal{L}_1$ error magnitude is divided by 2, while the $\mathcal{L}_0$ error magnitude is the same.

The identity function is thus impaired by a bias $\pm\frac{4}{3}\,\frac{M^3}{\omega'^2}$ reducing the output value with respect to the input value.

Assuming values are normalized, in the $[-1, +1]$ interval or in the $[0, 1]$ interval, we obtain, for the $\mathcal{L}_0$ error magnitude:

ω'	10	50	100	200	500	1000
$\epsilon_{l,\omega'}(1)$	0.0131	$0.533\,10^{-3}$	$0.133\,10^{-3}$	$0.333\,10^{-4}$	$0.54\,10^{-7}$	$0.12\,10^{-7}$

with a negligible error for, say, $\omega' \geq 100$.

At the implementation level we must introduce a change of variable, and consider:

$$\mathrm{Jd}_{\omega'}(x) \stackrel{\mathrm{def}}{=} \mathrm{h}(x/\omega') \qquad \text{with} \left\{ \begin{array}{l} \mathrm{Id}_{\omega'}(x) = \omega'\,(\mathrm{Jd}_{\omega'}(x) - 1/2) \\ \mathrm{Jd}_{\omega'}(x) = \mathrm{Id}_{\omega'}(x)/\omega' + 1/2 \end{array} \right.$$

in order to define a proper neuronoid computation, in the form given as definition. It is obvious to propagate this change of variable in all the equations.

It must be noted that although continous variables stands in $[0, 1]$ because of negative weights in the linear combination of input, intermediate values may be negative, as considred here.

- *Any programmatoid computation involving the $Id(\cdot)$ function can thus be approximated by a neuronoid, with $\tau = 0$, up to an affine change of variable, and sufficiently large ω', and a residual decreasing in a square way.*

Derivations are available as `Maple` code[16].

Collecting the previous results, we obtain:

- *Any programmatoid computation can be approximated, up to any precision, by a neuronoid computation.*

4.5 Approximation of switch mechanisms

Given normalized floating point variables $v_n \in [-1, 1], n \in \{1, N\}$ and binary variables $b_n \in \{0, 1\}, n \in \{1, N\}$, conditional expressions and related mechanisms require the implementation of formula of the form[17]:

[15] https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/mollificatio.mpl.out.txt

[16] https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/linearapproximation.mpl.out.txt

[17] We easily verifies that:
- if $b_n = 0$, while $|v_n/\omega'| \leq 1/\omega' \ll 1$, the term $\omega'\,\mathrm{h}(v_n/\omega' + \omega\,b_n - \omega) \simeq \mathrm{h}(-\omega) \simeq 0$ up to $O\left(e^{-4\,w}\right)$ as derived previously, while
- if $b_n = 1$,the term $\omega'\,\mathrm{h}(v_n/\omega' + \omega\,b_n - \omega) = \omega'\,\mathrm{h}(v_n/\omega')$ corresponds to the approximation of the identity function, up to $O\left(\frac{1}{\omega'}\right)$ as derived previously.

$$
\begin{aligned}
v &= \sum_{n\in\{0,N\}} b_n\, v_n \\
&= \sum_{n\in\{0,N\}} \omega'\, \mathrm{h}(v_n/\omega' + \omega\, b_n - \omega) + O\left((\omega')^{-2}\right) + O\left(e^{-4\,\omega}\right)
\end{aligned}
$$

The key point here is that product of a continuous variable v_n by a binary variable b_n, acting as a kind of switch, is not implementable with pure programmatoid computation because only products with constant parametric values, or weights. Up to our best understanding product between two variables, one being continuous can not be defined with lineat combinations unless additional mechanisms are introduced, as discussed below.

We also notice that the $\mathrm{Id}(\cdot)$ function mollification approximation is less efficient that the $\mathrm{H}(\cdot)$ function mollification, the former being subject to a polynomial remainder, the latter to an exponential one.

This means that:
- *Neuronoid computation allows one to implement more mechanism approximations, that exact programmatoid computation only.*

5 Examples of programmatoid computation

A step ahead, t seems obvious that we can designed temporizing mechanisms, oscillators and sequence generator, sleep sort mechanism, etc.

5.1 Memory gate

For a data input i, a control $i_l \in \{0,1\}$, and an output o, the equation:

$$o \leftarrow \text{if } i_l = 1 \text{ then } o \text{ else } i$$

implements, if $i \in \{0,1\}$, $o \in \{0,1\}$, a 1 bit memory, i.e., also called a RS gate, and reusing the previous conditional instruction parameters.

The key point is that it is now a recurrent system, which stability is obvious at the programmatoid level, but not necessarily at the neuronoid level, since we have an recurrent equation for $i_l = 1$.

For $i \in \{0,1\}$, $o \in \{0,1\}$ at the programmatoid level:

$$o \leftarrow \mathrm{H}(i - i_l - 1/2) + \mathrm{H}(o + i_l - 3/2)$$

For $i \in \{0,1\}$, $o \in \{0,1\}$:

$$
\begin{aligned}
o &\leftarrow \mathrm{h}(\omega\,(i - i_l - 1/2)) + \mathrm{h}(\omega\,(o + i_l - 3/2)) \\
&= O\left(e^{-4\,\omega}\right) + \mathrm{h}(\omega\,(o - 1/2))\Big|_{i_l=1}
\end{aligned}
$$

For $i \in [-1,1]$, $o \in [-1,1]$:

$$
\begin{aligned}
o &\leftarrow \omega'\,(\mathrm{h}(i/\omega' - \omega\, i_l) + \mathrm{h}(o/\omega' - \omega\,(1 - i_l)) \\
&= O\left(\omega'\, e^{-4\,\omega}\right) + \omega'\,\mathrm{h}(o/\omega')\Big|_{i_l=1}
\end{aligned}
$$

- The former equation is exact, so entirely stable during iterations.
- For the second equation, memorizing $o|_{t=0} \in \{0,1\}$, thus with $i_l = 1$, for

  $$\epsilon \overset{\text{def}}{=} \left\{ \begin{array}{cl} o & |_{o|_{t=0}=0} \\ 1 - o & |_{o|_{t=0}=1} \end{array} \right.$$

  we obtain[18]:

  $$
  \begin{aligned}
  \epsilon &\leftarrow (1 - 2\ o|_{t=0})\,\mathrm{h}(-(3/2 - i)\,\omega) + \mathrm{h}(\omega\,(\epsilon - 1/2)) \\
  &= \left\{ \begin{array}{lllll} 0 & \epsilon & = & 0 & \text{if } o|_{t=0} = 1 \\ e^{-2\,\omega} + O\left(e^{-4\,\omega}\right) & \epsilon & = & \varepsilon\, e^{-2\,\omega} & \text{otherwise} \end{array} \right.
  \end{aligned}
  $$

  In words : the iteration remains stable, and the error at the order of magnitude of $O\left(e^{-2\,\omega}\right)$. In particular the previous bias ε only impacts the $O\left(e^{-4\,\omega}\right)$ terms. Furthermore, if $o|_{t=0} = 1$, the positive

[18] For $o|_{t=0} = 0$ the derivation is straightforward. For $o|_{t=0} = 1$ and $i = 0$:

$$
\begin{aligned}
\epsilon &\leftarrow 1 - o \\
&= 1 - \mathrm{h}(-3/2\,\omega) - \mathrm{h}(\omega\,(1 - \epsilon + 1 - 3/2)) \\
&= 1 - \mathrm{h}(-3/2\,\omega) - \mathrm{h}(\omega\,(-\epsilon + 1/2)) \\
&= -\mathrm{h}(-3/2\,\omega) + \mathrm{h}(\omega\,(\epsilon - 1/2)) \qquad \text{since } \mathrm{h}(x) = 1 - \mathrm{h}(-x)
\end{aligned}
$$

with a similar derivation for $o|_{t=0} = 1$ and $i = 1$.

bias yields a convergence towards 1 without any bias if $i = 1$, and with a exponentially decreasing bias if $i = 0$.

- For the third equation, memorizing $o|_{t=0} \in [-1, 1]$, thus with $i_l = 1$, while $i \in [-1, 1]$, we obtain:

$$\begin{array}{rcl} o & \leftarrow & \omega' \, (\mathrm{h}(i/\omega' - \omega \, i_l) + \mathrm{h}(o/\omega' - \omega \, (1 - i_l)) \\ \multicolumn{3}{l}{\text{while, since } i_l = 1\text{:}} \\ & = & \omega' \, (\mathrm{h}(i/\omega' - \omega) + \mathrm{h}(o/\omega') \\ \multicolumn{3}{l}{\text{while, when neglecting } |i/\omega'| < 1/\omega' \ll \omega\text{:}} \\ & = & \omega' \, (\mathrm{h}(-\omega) + \mathrm{h}(o/\omega') \\ \multicolumn{3}{l}{\text{while, using previous series formula:}} \\ & = & o \pm \frac{4}{3} \frac{1}{\omega'^2} + e^{-4\omega} + O\left(\frac{1}{\omega'^4}\right) + O\left(e^{-8\omega}\right) \\ \multicolumn{3}{l}{\text{while, considering only the larger term:}} \\ & \simeq & o \pm \frac{4}{3} \frac{1}{\omega'^2} = o|_{t=0} \pm t \, \frac{4}{3} \frac{1}{\omega'^2} \end{array}$$

  The memorization is thus impaired by a linear bias only, thanks to the fact that linear unit has been normalized. Because $O\left(e^{-4\omega}\right) \ll O\left(\frac{1}{\omega'^2}\right)$, fro the chosen values, the bias due the use of a neuronoid approximation of a programmatoid is negligible with respect to the bias due to the use of a neuronoid approximation of a linear unit.

Derivations are available as `Maple` code[19].

5.2 Multi-stable mechanisms

The previous developments leads to solutions for several temporal systems, as briefly given now.

5.2.1 Bi-stable mechanisms

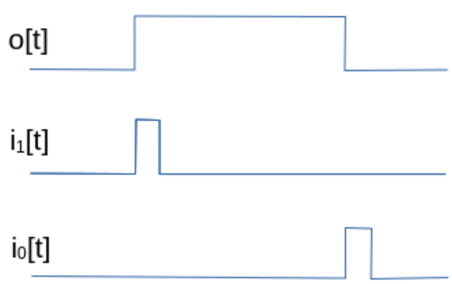


Figure 2: With:

$$\begin{array}{rcl} o & \leftarrow & \text{if } o = 0 \text{ and } i_1 = 1 \text{ then } 1 \\ & & \text{elif } o = 1 \text{ and } i_0 = 1 \text{ then } 0 \\ & & \text{else } o \end{array}$$

a bi-stable system with two 1 and 0 inputs, is implemented.

Figure 3: With:

$$\begin{array}{rcl} o & \leftarrow & \text{if } o = 0 \text{ and } i_1 = 1 \text{ then } 1 \\ & & \text{elif } o = 1 \text{ and } i_0 = 1 \text{ then } 0 \\ & & \text{else } o \\ i_1 & \leftarrow & \text{if } i_1 = 0 \text{ and } i = 1 \text{ then } 1 \\ & & \text{elif } o = 0 \text{ then } 0 \\ & & \text{else } i_1 \\ i_0 & \leftarrow & \text{if } i_0 = 0 \text{ and } i = 0 \text{ then } 1 \\ & & \text{elif } o = 1 \text{ then } 0 \\ & & \text{else } i_0 \end{array}$$

a bi-stable system with one two-way switch is implemented, using internal variables i_0 and i_1. This corresponds to a frequency divider by 2.

[19] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/memorygate.mpl.out.txt`

5.2.2 Spike generation

Figure 4: With:

$$
\begin{array}{lcl}
o & \leftarrow & \text{if } r = 0 \text{ and } i = 1 \text{ then } 1 \\
 & & \text{else } 0 \\
r & \leftarrow & \text{if } o = 1 \text{ then } 1 \\
 & & \text{elif } i = 0 \text{ then } 0 \\
 & & \text{else } r
\end{array}
$$

a spike generation detecting signal rising, with an internal variable r registering if already detected.

5.2.3 Delay

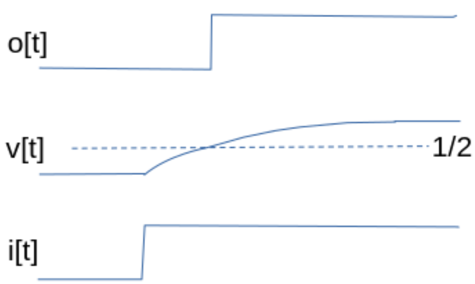


Figure 5: With:

$$
\begin{array}{lcl}
o & \leftarrow & \text{if } v > 1/2 \text{ then } 1 \text{ else } 0 \\
v & \leftarrow & (1-\gamma)\, v + \gamma\, i
\end{array}
$$

a delayed output is implemented. Assuming that $v = 0$, when $i = 1$ and remains at this value, we obtain for the delay T:

$$T = -\frac{\log(2)}{\log(1-\gamma)} > 0 \Leftrightarrow \gamma = 1 - 2^{-\frac{1}{T}}.$$

Here we assume that $i = 1, t \in [0, T]$ but it is very easy to avoid this constraint with a bi-stable mechanism as input.

Derivations are available as `Maple` code[20].

Delay on rising front

- i_1 starts the delay
- i_0 resets the state and the output

$$
\begin{array}{lcl}
o & \leftarrow & \text{if } v > 1/2 \text{ then } 1 \text{ else } 0 \\
v & \leftarrow & \text{if } i_0 = 1 \text{ then } 0 \text{ else } (1-\gamma)\, v + \gamma\, s \\
s & \leftarrow & \text{if } s = 0 \text{ and } i_1 = 1 \text{ then } 1 \\
 & & \text{elif } s = 1 \text{ and } i_0 = 1 \text{ then } 0 \\
 & & \text{else } s
\end{array}
$$

Time bounded output

- i_1 starts the bounded time output
- i_0 resets the state and the output

$$
\begin{array}{lcl}
o & \leftarrow & \text{if } v < 1/2 \text{ and } s = 1 \text{ then } 1 \text{ else } 0 \\
v & \leftarrow & \text{if } i_0 = 1 \text{ then } 0 \text{ else } (1-\gamma)\, v + \gamma\, s \\
s & \leftarrow & \text{if } s = 0 \text{ and } i_1 = 1 \text{ then } 1 \\
 & & \text{elif } s = 1 \text{ and } i_0 = 1 \text{ then } 0 \\
 & & \text{else } s
\end{array}
$$

[20] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/delayastable.mpl.out.txt`

5.2.4 Oscillation

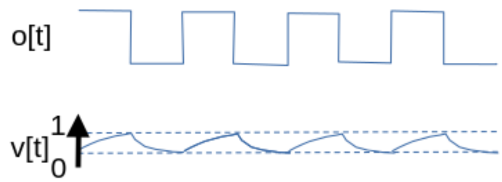


Figure 6: With:

$$\begin{array}{rcl} o & \leftarrow & \text{if } v < 1/3 \text{ then } 0 \\ & & \text{elif } v > 2/3 \text{ then } 1 \\ & & \text{else } o \\ v & \leftarrow & (1-\gamma)\, v + \gamma\, (1-o)\, i \end{array}$$

an binary oscillator is implemented. It is stopped if $i = 0$ and running for $i = 1$. We obtain for the period T:

$$T = -\frac{2\,\log(2)}{\log(1-\gamma)} > 0 \Leftrightarrow \gamma = 2 - 2^{1-\frac{1}{T}}.$$

Derivations are available as `Maple` code[21].

6 Examples of neuronoid computation

6.1 The explog function

The maximal and the average operators can be easily combined. For instance, given K inputs $i_k \in [-1, 1], k \in \{1, K\}$, let us define:

$$\begin{array}{rcll} o & \stackrel{\text{def}}{=} & \frac{1}{\mu} \log\left(\frac{1}{K} \sum_{k \in \{1 \cdots K\}} \exp(\mu\, i_k)\right) & \\ & = & \frac{1}{K} \sum_k i_k + O(\mu) & \text{(average)} \\ & = & \max_k(i_k) - \frac{\log(K)}{\mu} + \frac{O\left(e^{-\nu\,\mu}\right)}{\mu}, \nu > 0 & \text{(max)} \\ & = & i_1|_{i_i = i_2 \cdots = i_K} & \\ & \in & [-1, 1] & \end{array}$$

with $\mu > 0$ parameterizing the balance between:

- the average, for small μ, line 2 above,
- the maximum, for large μ, line 3 above, up to an offset $-\log(K)/\mu$,

as shown in Figure 7.

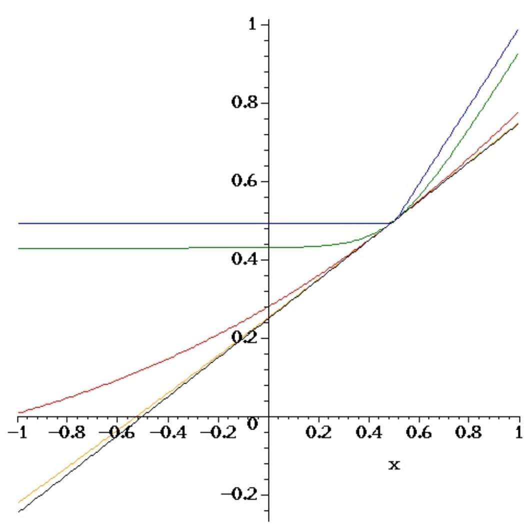


Figure 7: Representation of the exp-log function for $K = 2$, $i_2 = 1/2$, with $i_1 \in [0, 1]$ and $\mu \in \{0.01, 0.1, 1, 10, 100\}$, from top to bottom, in black, orange, red, green and blue, respectively. With small μ it is closed to linear average, whereas for $\mu = 100$ it is close to $\max(x, 1/2)$.

[21] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/delayastable.mpl.out.txt`

From Figure 7, we observe that for $i_k \in [-1, 1]$, $\mu \in [0.01, 100]$ is a reasonable range to approximate the different profiles, so that:

- The $\exp(\cdot)$ function range is in $[-100, 100]$.
- The $\log(\cdot)$ function range is in $[e^{-100} \simeq 1, e^{100}]$.

These are huge ranges but we may consider the following renormalization:

$$\begin{array}{rcl} i_k & \rightarrow & \frac{i_k}{\mu} \\ o & \rightarrow & \frac{o}{\mu} \\ o & = & \log\left(\frac{1}{K} \sum_{k \in \{1 \cdots K\}} \exp(i_k)\right) \end{array}$$

so that the $\exp(\cdot)$ range is now $[-1, 1]$ and the $\log(\cdot)$ range is now $[1, e]$, allowing an implementation with not so big function's range. This is fine, providing that we can implement the $\exp(\cdot)$ and the $\log(\cdot)$ functions with a sufficient numerical precision, because, we now are working with small values of i_k.

Derivations of this section are available as `Maple` code[22].

6.2 Approximation of `exp` and `log`

Taking benefit of the fact that sigmoid is convex as the exponential function in some range, and concave as the logarithm function in some range, we can easily design the following approximations, obtained by some manual intuitive choice of the sigmoid combination and numerical adjustment of the parameters. Two examples are given in Figure 8 and Figure 9. We share these examples to illustrate fact we really can approximate several bounded non-linear function with neuronoid, but we will use another track, to

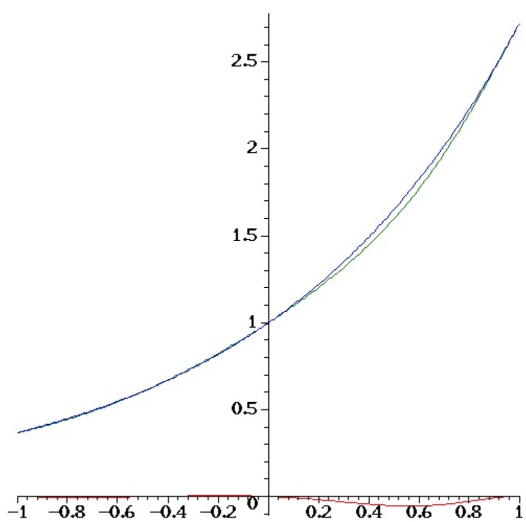


Figure 8: Approximation in the $[-1, 1]$ interval of the exponential function, in blue, by a sigmoid combination, in green:

$$0.182 + 2.34\,\mathrm{h}(x - 1) + 1.55\,\mathrm{h}(x/2):$$

yielding a $\mathcal{L}_1$ error of 0.01, the error being drawn in red.

[22] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/explog.mpl.out.txt`

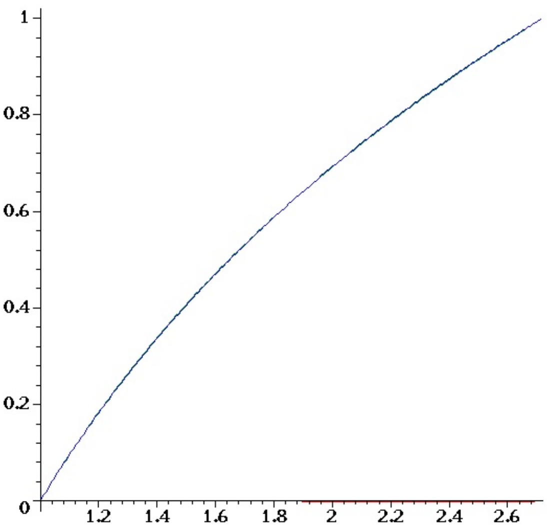


Figure 9: Approximation in the $[1, e]$ interval of the logarithm $\log(x)$ function, in blue, by a sigmoid combination, in green:

$$-5.10 + 2.60\,\mathrm{h}(x/2) + 4.70\,\mathrm{h}(x/10) :$$

yielding a $\mathcal{L}_1$ error of 0.001, the error being drawn in red.

Given these two approximations, we can implement an approximation of the renormalized `explog` function proposed previously, with $N_l = 1$ in this case.

Derivations of this section are available as `Maple` code[23].

6.3 Approximation of valued product.

Let us now consider two values $v_i \in [-1, 1], i \in \{1, 2\}$, and apply the basic formula:

$$\exp(\log(v_1) + \log(v_2)) = v_1\, v_2$$

in such a way that the the log() and exp() functions are used within the ranges defined previously. We obtain:

$$\begin{array}{rcll}
l(v_i) & \stackrel{\text{def}}{=} & \frac{e-1}{2}\, v_i + \frac{e+1}{2} & \in \; [1_{v_i=-1}, e_{v_i=1}], \\
l(v_1, v_2) & \stackrel{\text{def}}{=} & \log(l(v_1)) + \log(l(v_2)) - 1 & \in \; [-1, 1], \\
b & \stackrel{\text{def}}{=} & \frac{4}{(e-1)^2} & c \; \stackrel{\text{def}}{=} \coth\left(\frac{1}{2}\right) \\
p(v_1, v_2) & \stackrel{\text{def}}{=} & b\, e^{l(v_1, v_2)} - c\,(v_1 + v_2 + c) & \\
& = & v_1\, v_2 &
\end{array}$$

Therefore, valued products can be approximated by neuronoids. This for instance quite useful for mechanism of gain control or using adaptive mechanisms, as for instance a softmax mechanism discussed now.

Derivations of this section are available as `Maple` code[24].

6.4 Approximation a softmax operator

Another track is to consider the programmatoid implementation of the maximum operator, given K inputs $i_k \in [-1, 1], k \in \{1, K\}$, as a weighted sum:

$$o \stackrel{\text{def}}{=} \max_k(i_k) = \sum_k b_k\, i_k$$

with:

$$b_k \stackrel{\text{def}}{=} And_{l \in \{1, k-1\}} \mathrm{H}(i_k > i_l) \text{ and } And_{l \in \{k+1, K\}} \mathrm{H}(i_k \geq i_l),$$

So that $b_k = 1$ if i_k is the first instantiating of the max value [25], thus with a form similar to the average operator $\mathrm{mean}_k(i_k) = 1/K \sum_k b_k\, i_k$.

We easily derive:

[23] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/explog.mpl.out.txt`

[24] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/explog.mpl.out.txt`

[25] First, assume that all i_k are different; then $b_k = 1$ if and only if $\forall l \neq k, i_k > i_l$, yielding the expected result.
Then, if more than one value corresponds to $\max_k(i_k)$, let us consider $k_0 = \; \mathrm{argmin}\{k, k = \max_k(i_k)\}$.
For k_0, $And_{l \in \{k_0+1, K\}} \mathrm{H}(i_{k_0} \geq i_l) = 1$ because it is a maximum and $And_{l \in \{1, k_0-1\}} \mathrm{H}(i_{k_0} > i_l) = 1$ because no previous maximum is encountered, by definition. Then for subsequent values $k_1 = \max_k(i_k)$, because there is $k_0 < k_1$ with $i_{k_0} = i_{k_1}$, then $And_{l \in \{1, k_1-1\}} \mathrm{H}(i_{k_1} > i_l) = 0$, so that we only have one occurrence of i_k which is selected.

$$\begin{array}{rcl} b_k & \stackrel{\rm def}{=} & And_{l \in \{1,k-1\}} \mathrm{H}(i_k > i_l) \mbox{ and } And_{l \in \{k+1,K\}} \mathrm{H}(i_k \geq i_l) \\ \multicolumn{3}{l}{\mbox{while, expanding the } And \mbox{ operator}} \\ & = & \mathrm{H}\left(\sum_{l \in \{1,k-1\}} \mathrm{H}(i_k > i_l) + \sum_{l \in \{k+1,K\}} \mathrm{H}(i_k \geq i_l) - K + 3/2\right). \end{array}$$

Then, introducing a linearly combination of both operators:

$$\begin{array}{rcl} o & \stackrel{\rm def}{=} & g\,\left(\frac{1}{K} \sum_k i_k\right) + (1-g)\,\left(\sum_k b_k\, i_k\right) \\ & = & \sum_k \frac{g}{K}\, i_k + \sum_k b_k\, ((1-g)\, i_k), \end{array}$$

we have constructed a softmax mechanism, directly based on the previous developments, without introducing numerical approximations of other non-linear functions, as shown in Figure. 6. The formula can also be mollified if required. It is a correct programmatoid mechanism, because g is a constant, so we only have either product with a constant or with a binary value.

We also could have considered g as an input value, yielding product of two valued inputs, thus involving mechanisms discussed previously.

Derivations of this subsection are available as `Maple` code[26].

7 Comparing the $h(\cdot)$ sigmoid with other non linearity

We have $\mathrm{h}(x) = \frac{1+\tanh(2\,x)}{2}$ thus based on the `tanh` non linearity. This corresponds most common activation function when deriving a rate-based reduced neural network from bio-physical elements [Cessac and Samuelides, 2007]. This is far from being the only choice, as reviewed in [Szandała, 2020], and we could have considered, for instance $\bar{h}$ functions of the form:

Name:	Tanh	Erf	Arctan	Softsign
Formula:	$\frac{1}{2} + \frac{\tan\mathrm{h}(2\,x)}{2}$	$\frac{1}{2} + \frac{\arctan(\pi\,x)}{\pi}$	$\frac{1}{2} + \frac{\mathrm{erf}(\sqrt{\pi}\,x)}{2}$	$\frac{1}{2} + \frac{x}{1+\lvert 2\,x\rvert}$
Sharpness:		$\lvert\mathrm{h}(x)\rvert < \lvert\bar{h}(x)\rvert$	$\lvert\mathrm{h}(x)\rvert > \lvert\bar{h}(x)\rvert$	$\lvert\mathrm{h}(x)\rvert > \lvert\bar{h}(x)\rvert$
$\mathcal{L}_1 =$	0	$-\frac{\pi\,\log(2)-2}{2\,\pi}$	$+\infty$	$+\infty$
$\mathcal{L}_0 \simeq$	0	0.018	0.082	0.81
κ_{max}	1.53	1.52	2.04	4

while all profiles are normalized and symmetric, i.e.:

$$\bar{h}(-\infty) = 0, \bar{h}(0) = 1/2, \bar{h}'(0) = 1, \bar{h}(+\infty) = 1, \bar{h}(x) = 1 - \bar{h}(-x)$$

and are drawn in the Figure 10.

- The Arctan and Softsign profiles are smoother than the Tanh profile, whereas we are looking here for sharper profiles in order to better approximate the step-function. They also have higher maximal curvatures κ_{max} which means that the curvature is more inhomogeneous than for the Tanh profile, and they do not correspond to biologically plausible activation functions [Cessac and Samuelides, 2007], despite the fact they could be of great interest in machine learning [Szandała, 2020].

- The error function[27], Erf, based profile is very closed numerically from the Tanh profile, with a finite $\mathcal{L}_1 \simeq -0.028$ small distance magnitude, and a $\mathcal{L}_0 < 2\%$ small distance magnitude, with similar maximal curvatures κ_{max} up to 1%, and is also quoted as a biologically plausible activation functions [Cessac and Samuelides, 2007]. Though its sharpness is a bit better than for the Tanh profile, with a better distribution of the curvature, all algebraic derivations made in this paper would not have as easy as its is now.

[26] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/softmax.mpl.out.txt`

[27] `https://en.wikipedia.org/wiki/Error_function`

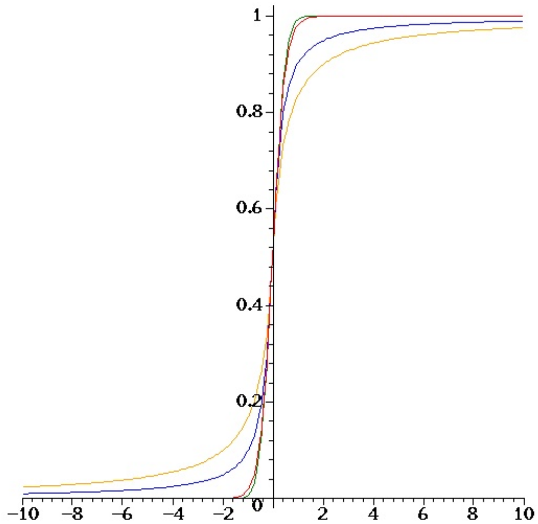


Figure 10: Comparison between the normalized Tanh profile in red, the Erf profile in green, the Arctan profile in blue and the Softsign in orange, i.e., from the sharpest to the smoothest.

Derivations are available as `Maple` code[28].

8 Using the braincraft challenge setup

This documentation allows to use the braincraft challenge for a programmatic implementation, or for a programmatoid/neuronoid implementation using code translator (not yet available).

Here is braincraft challenge original documentation original documentation[29].

You must be familiar with basic `git` usage and basic `python` programming.

8.1 Installation of the setup

1. Connect to `https://github.com` with your login.
2. Go to the braincraft challenge[30] and create a new fork:


3. Download the repository in SSH read/write mode:


4. In the braincraft local git directory, run
`make demo`

[28] `https://raw.githubusercontent.com/vthierry/braincraft/master/doc/tex/sigmoidatan.mpl.out.txt`
[29] `https://github.com/rougier/braincraft/blob/master/README.md`
[30] `https://github.com/vthierry/braincraft`

Note: You may have to install:

```
sudo apt install python3-tqdm
```

while using a virtual environment, is advised, but not mandatory.

8.2 Running at the programmatic level

At the programmatic level, the bot behavior is directly programmed implementing a `State` in the target programming language:

```
# Implements a programmatoid state.
class State:
  # The state data: dictionary.
  #  Stores all input, output, and internal variables, including parameters.
  data = dict()
  # Inits some data, if needed.
  def init(self):
  # Updates, at each step, the data from input and sets the output value.
  def update(self):
```

The name `programmaticoid_...` corresponds to a programmatic implementation in the target programming language, but using only programmatoid constructs. It is used for testing purposes.

In `Python` one can start from the following example:

`programmatic_task1.py`[31]

The `programmatoid_challenge.py`[32] source file includes all documented methods and the related API documentation is available here:

`programmatoid challenge API`[33]

8.3 Running at the programmatoid level

At the programmatoid level, the system is defined by a set of equations of the form:

```
# A tiny example of programmatoid
subs( ## Fixed parameters sequence of constant values
    T = 10, ### The delay
    constantName = constantValue  ### Any other constants
,
[  ## The package options
  prgm_options = { omega = 10 },
  ## The package equations
  Delay(b, i_t, T),
  a = H(H(b))
])
```

Note: The equation list being evaluated in sequence, equations with input must precede equations with output.

This piece of `Maple` code, stored in a file of the form `prgm_name.mpl` is compiled in the form of straight-line program, with a programmatoid syntax, as defined in this documents.

It is output as a piece of `Python` code and then executed in the desired environment using the:

`programmatoid challenge script`[34]

`Usage: A COMPLETER QUAND STABLE`

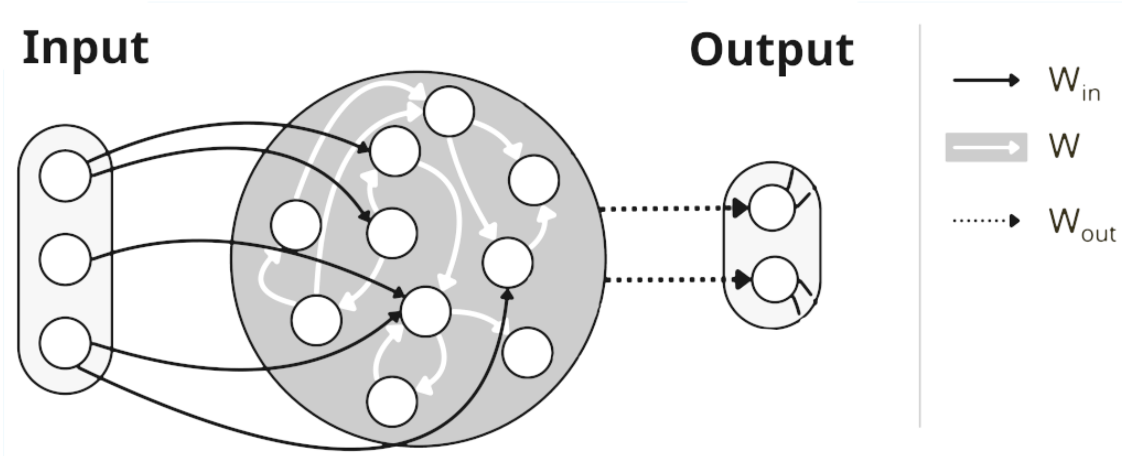
The compilation is parameterized by a set of options given in appendix B.1 and is defined using a set of functions given in appendix B.2 corresponding to all functionnalities defined in this document.

The `./out/$prgm/test.out.wjson` output file format uses a dialect of the JSON[35] syntax, called `wJSON`, for weak-json as presented here[36].

[31] https://github.com/vthierry/braincraft/blob/master/braincraft/programmatic_task1.py
[32] https://github.com/vthierry/braincraft/blob/master/braincraft/programmatoid_challenge.py
[33] https://html-preview.github.io/?url=https://github.com/vthierry/braincraft/blob/master/doc/api/programmatoid_challenge.html
[34] https://github.com/vthierry/braincraft/blob/master/braincraft/programmatoid_challenge.sh
[35] https://www.json.org/json-en.html
[36] https://line.gitlabpages.inria.fr/aide-group/wjson/

8.4 Running at the neuronoid level

If the `network_weights` compilation option is raised, is is compiled in the form of a network weights as illustrated[37] here:



and run using the specific recurrent equations, on input $\mathbf{I}$ and output $\mathbf{O}$, with internal values $\mathbf{X}$:

$$\mathbf{X} = h\left(\mathbf{W}\,\mathbf{X} + \mathbf{w} + \mathbf{W}_{in}\,\mathbf{I}\right), \quad \mathbf{O} = \mathbf{W}_{out}\,\mathbf{X}, \quad \mathbf{X}|_{t=0} = \mathbf{0}$$

where:
- $W_{out}^{i,j} = \delta_{i=j}$, in words, the output is just a sub-vector of the internal values.
- There is no leak, but recurrent connections acting as discrete implementation of a leak.
- There is an offset $\mathbf{w}$ that equivalently corresponds to a a input weight on a constant input.
- The non-linearity is a normalized sigmoid.

In fact this compilation option is simply used to verify numerically that such a network is totally equivalent to programmatoid implementation with only neuronoids.

References

[Cessac and Samuelides, 2007] Cessac, B. and Samuelides, M. (2007). From neuron to neural networks dynamics. *The European Physical Journal Special Topics*, 142(1):7–88.

[Lapicque, 1907] Lapicque, L. (1907). Recherches quantitatives sur l'excitation electrique des nerfs traitee comme une polarization. *Journal de physiologie et de pathologie générale*, 9:620–635.

[Szandała, 2020] Szandała, T. (2020). *Review and Comparison of Commonly Used Activation Functions for Deep Neural Networks.*

A An alternative solution without about-turn

We have proposed half-turn mechanism, because the implementation is relatively simpler, thus clearer, for benchmarking the programmatoid and neuronoid implementations. However we also can propose a quarter-turn implementation.

A.0.1 Navigation

Heuristic : The bot runs at constant velocity,

(i) ahead by default, thanks to a linear correction, high enough to correct the direction, small enough to avoid oscillations, and

(ii) perform a quarter-turn in the preferred orientation as soon as a wall is closed ahead, or if in the middle of the corridor.

- With this navigation mechanism the bot performs either leftward or rightward half-loops, traversing the central corridor.
- In fact, the situation is quite complex because the sensors do *not* detect any crossing, because the field of view is too narrow. Therefore, predefined delays are to be used, in order to detect tthe middle of the downward corridor.

[37] Shared here by courtesy of R. de Coudenhove, Y. Bendi-Ouis, A. Strock and X. Hinaut.

Internal constants

γ	$=$	0.035	Linear feedback gain in forward mode, maximize stability without oscillation.
α	$=$	5.0	Maximal turn angle, allowing to perform a quater turn in 18 time steps.
β_1	$=$	0.8	Proximity threshold triggering a quater-turn.
β_0	$=$	0.7	Proximity threshold ending a quater-turn.
δ_1	$=$	18	Quarter-turn duration in number of time steps.
δ_0	$=$	8	Delay between two quarter-turns in number of time steps.

Internal variable

$b_d \in \{0_{\text{rightward}}, 1_{\text{leftward}}\}$,	$b_d\|_{t=0} = 0$	Quarter-turn direction, either left (1) or right (0).
$b_t \in \{0_{\text{forward}}, 1_{\text{turning}}\}$,	$b_t\|_{t=0} = 0$	Forward (0) versus turning (1) mode.
$b_{t\bullet} \in \{0, 1\}, \bullet \in \{0, 3\}$	$b_{t\bullet}\|_{t=0} = 0$	Turn 0, 1, 2 or 3 mode.
$b_{t30} \in \{0, 1\}$	$b_{t30}\|_{t=0} = 0$	Turn 3 waiting delay before turing.
$g_{e\bullet} \in \{0, 1\}, \bullet \in \{0, 1\}$	$g_{e\bullet}\|_{t=0} = 0$	Energy after and before turning.
$b_{3w\bullet} \in \{0, 1\}, \bullet \in \{0, 1\}$	$g_{e\bullet}\|_{t=0} = 0$	Delay variables for turrn 3 start and stop.

Implementation

```
# Starts turning because of the wall distance
if b_t == 0 and b_t30 == 0 and b_t3 == 0 and p_a > 0.8
  b_t = 1
  # Counts the turns
  if b_t0 == 0 and  b_t1 == 0 and b_t2 == 0
    b_t0 = 1
  elif b_t1 == 0 and b_t2 == 0
    b_t1 = 1
    b_t0 = 0
  elif b_t2 == 0
    b_t2 = 1
    b_t1 = 0
   # Stores energy before and after a corridor
   if b_t2 == 1
    g_e0 = g_e
  elif b_t1 == 1
    g_e1 = g_e
  elif b_t0 == 1
    g_e0 = 0
    g_e1 = 0
    endif
# Stops turning because of the wall distance
elif b_t == 1 and b_t30 == 0 and b_t3 == 0 and p_a < 0.7
  b_t = 0
endif
# Updates time delay variables
Delay_0(b_t3w0, b_t == 0 and b_t30 == 0 and b_t3 == 0 and b_t2 == 1, 8)
Delay_0(b_t3w1, b_t30 == 1 and b_t3w0 == 1, b_t3w1, 18)
# Starts and stops turning because of a left crossing in the middle of the corridor
if b_t == 0 and b_t30 == 0 and b_t3 == 0 and b_t2 == 1
  b_t2 = 0
  b_t30 = 1
elif b_t30 == 1 and  b_t3w0 == 1
  b_t = 1
  b_t3 = 1
  b_t30 = 0
elif b_t3 == 1 and b_t3w1 == 1
  b_t = 0
  b_t3 = 0
endif
# Changes direction if energy did not increase
if  g_e0 <  g_e1
```

```
b_d = 1 - b_d
endif
```

with the same equations to control the `d_l` and `d_r` output variables as before. The behavior is tested in programmatic_quarter_turn_task1.py[38].

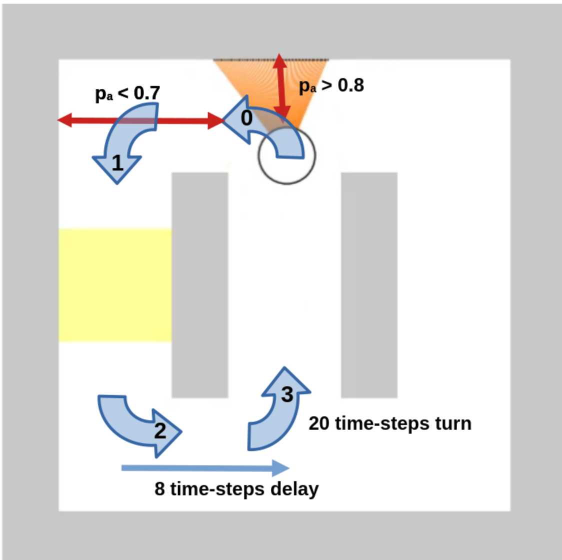


Figure 11: Illustrates the quarter turn implementation: it starts when the ahead proximity to a wall exceeds a threshold, and has a predefined duration, using a timer.

Figure 12: Illustrates the navigation implementation. The navigation is a loop made of 4 quarter-turns.
-Turns 0, 1 and 2 are triggered by the wall ahead proximity.
-Turn 3 is triggered by a delay after turn 2 and has a predefined precalibrated duration.

[38]https://raw.githubusercontent.com/vthierry/braincraft/master/braincraft/programmatic_quarter_turn_task1

B Programmatoid package reference

B.1 Package and compiler options and outputs

Name	Format	Value	
Package meta-data.			
`file`	string	`path/name`	The package source path name, without the '.mpl' extension.
`name`	string	from file name	The package name.
`excerpt`	string	...	A one or two lines description.
`homepage`	URL	...	A link to the package, e.g., a git page.
`version`	date		The compilation date and time.
`license`	string	`CeCILL-C` or ...	The share license.
`author`	Name <email>	...	The person to contact.
Compiler parameters.			
`omega`	large float	1000	The ω used for `Id()` and `H()` mollification.
`mollification`	Boolean	false	Whether `Id()` and `H()` functions are converted to `h()`.
`networking`	Boolean	false	Whether no code, but network's weights is generated.
`python`	Boolean	true	If defined, generates a Python piece of code of name `update_state` with the flattened code.
`inputs`	list(name)	r.h.s. variables not in l.h.s.	Input variables, user defined or deduced.
`outputs`	list(name)	l.h.s. variables not in r.h.s.	Input variables, user defined or deduced.
Evaluation parameters.			
`challenge`	int		Challenge number 1, 2, or 3; mandatory
`timeout`	int	0	Maximum number of iterations, no bound if 0.
`runs`	int	1	Number of runs, for the evaluation.
`display`	Boolean	true	Whether to display animation or not.
`rate`	float	0	Delay in second between two iterations (used for display).
Compiler output.			
`error`	string	... or not :)	The error message, if any
`source`	string		The parsed input source, without the user options.
`expanded`	string		The parsed source, with known functions expanded.
`flattened`	string		The expanded source, with all inner functions generated by new equations.
`network[dimension]`	int		The network internal unit count.
`network[connectivity]`	int		The network non-sero weight count.
`network[f]`	list(name)		The unit's non-linearities.
`network[W_in]`	matrix		The input weight's matrix.
`network[W]`	matrix		The recurrent weight's matrix.
`network[w]`	matrix		The recurrent weights offset, or bias.
`network[W_out]`	matrix		The output weight's matrix.

Note: All package meta-data and compiler parameters can be overwritten in the source `prgm_options` first-line.

B.2 Programmatoid functions

For binary variables `b_•` $\in \{0, 1\}$, continuous variables `v_•` $\in [-1, 1]$, and the sigmoid $h(\cdot)$ the following functions, defined previously, are implemented:

`v_o = h(v)`	The normalized sigmoid.
`v_o = H(v)`	Generalized step-function.
	`v_o` $\in \{0_{v<0}, 1/2_{v=0}, 1_{v>0}\}$
	Converted to `h()` when the option `prgm["neuronoid"] = true`.
`v_o = Id(v)`	Identity function
	Converted to a neuronoid when the option `prgm["neuronoid"] = true`.
`b_o = And(b_1, ...)`	Conjunction with a variable number of binary arguments.
	Implements `b_1 and b_2 ...`
`b_o = Or(b_1, ...)`	Disjunction with a variable number of binary arguments.
	Implements `b_1 or b_2 ...`
`b_o = Not(b)`	Negation of a binary value.
	Implements `not b`
`b_ o = If_b(b_1, b'_1, ..., b'_0)`	Binary conditional expression,
	Implements `if b_1 then b'_1 ....`
`v_o = If_v(b_1, v_1, ..., v_0)`	Valued conditional expression,
	Implements `if b_1 then v_1 ....`
`v_o = Exp_v(v_1)`	Exponential function approximation
	Implements $v_1 \in [-1, 1], e^{v_1} \in [1/e, e] \simeq [0.368, 2.718]$
`v_o = Log_v(v_1)`	Natural logarithm approximation
	Implements $v_1 \in [1, e], \log(v_1) \in [0, 1]$
`v_o = Prod_v(v_1, v_2)`	Multiplication approximation
	Implements $v_i \in [-1, 1], i \in \{1, 2\}, v_1\, v_2 \in [-1, 1]$
`v_o = Prod_b(b_1, v_1, ...)`	Sum of binary weights values
	Implements `b_1   v_1 + ....`
	Default missing value is 0.
`v_o = Softmax(v_1, ..., G)`	Mean-max operator.
	`G` $\in [0_{maximum}, 1_{average}]$ controls the mean-max balance.
`Latch_b(b_o, b_i, b_c)`	Binary memory latch.
	`b_o` is the output,
	`b_i` is the input,
	`b_c` $\in \{0_{open}, 1_{latched}\}$ is the control.
`Latch_v(b_o, v_i, b_c)`	Valued memory latch.
	`b_o` is the output,
	`v_i` is the input,
	`b_c` $\in \{0_{open}, 1_{latched}\}$ is the control.
`Bistable(b_o, b_1, b_0)`	Two inputs bi-stable latch.
	`b_o` is the output,
	`b_0` resets to 0,
	`b_1` sets to 1.
`Bistable(b_o, b_i)`	One input bi-stable latch.
	`b_o` is the output,
	`b_i` is the two-way input.
`Spikeup(b_o, b_i)`	Spike generation on input rising front.
	`b_i` is the input, which rising front is detected.
`Delay(b_o, b_i, T)`	Basic delayed output.
	`T` > 0 is the delay, in number of global clock event,
	`b_o` is the output,
	`b_i` is the input, that must remains to 1.
`Delay_0(b_o, b_1, b_0, T)`	Delayed output.
	`T` > 0 is the delay, in number of global clock event,
	`b_o` is the output,
	`b_1` is the input,
	`b_0` is the optional reset.
`Delay_1(b_o, b_1, b_0, T)`	Time bounded output.
	`T` > 0 is the delay, in number of global clock event,
	`b_o` is the output,
	`b_1` is the input,
	`b_0` is the optional reset.
`Oscillator(b_o, b_c, T)`	Oscillatory output.
	`T` > 0 is the period, in number of global clock event.
	`b_o` is the output,
	`b_c` $\in \{0_{stop}, 1_{run}\}$ is the control.